



Передовые
инженерные
школы



МИЭТ

НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Передовая инженерная школа «Средства проектирования и
производства электронной компонентной базы»

Магистерская программа «Вычислительные системы и ЭКБ»

Дисциплина:

«Маршрут проектирования цифровых интегральных схем»

Лабораторная работа 02 – Логический синтез и проверка эквивалентности

Кузьмин Павел Андреевич

2025 год

Освоить синтез RTL в библиотеку стандартных ячеек и проверить эквивалентность полученной схемы исходному RTL.

01 Логический синтез

Подготовить управляющий Tcl-скрипт и выполнить синтез в Cadence Genus.

02 Анализ реализации

Изучить состав ячеек, площадь и временные характеристики по отчётам.

03 Проверка эквивалентности

Выполнить LEC в Cadence Conformal и исследовать намеренно внесённую ошибку.

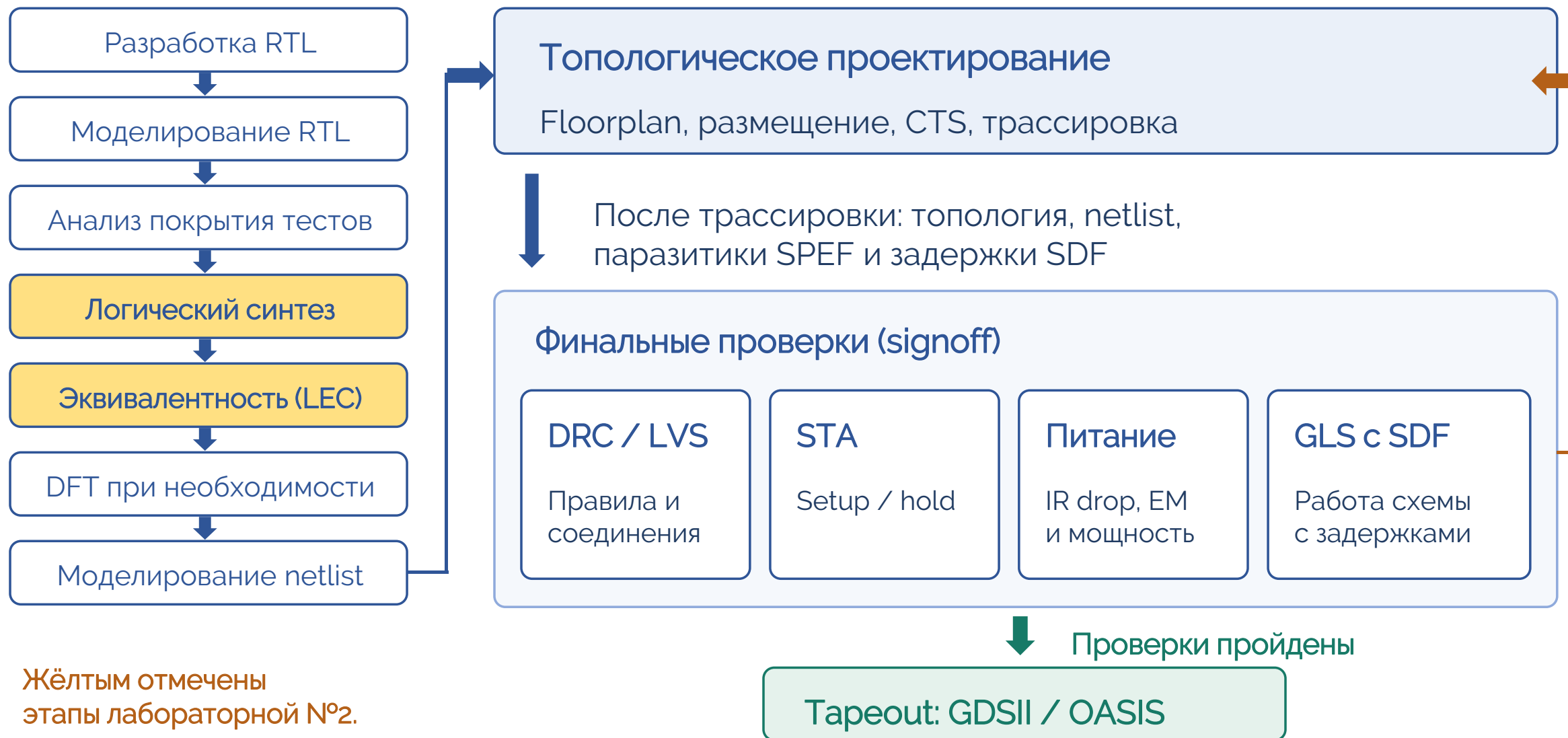
04 Расчёт по библиотеке .lib

Разобрать таблицы и рассчитать задержки и выходные transition ячеек.

Результат: netlist, прошедший LEC, для дальнейшего топологического проектирования.



Маршрут RTL-to-GDS



Жёлтым отмечены
этапы лабораторной №2.

При нарушениях исправляем проект и повторяем проверки.



Преобразует RTL-описание в netlist стандартных ячеек с сохранением функционального поведения

RTL-описание

```
assign y = ~(a & b);
```



Cadence Genus

Оптимизация логики
Выбор ячеек
Оптимизация реализации



Netlist

```
NAND2_X1 u1 (  
    .A(a), .B(b),  
    .Y(y)  
);
```



Библиотека .lib



Ограничения SDC

Выбор реализации с учётом временных ограничений, площади и энергопотребления

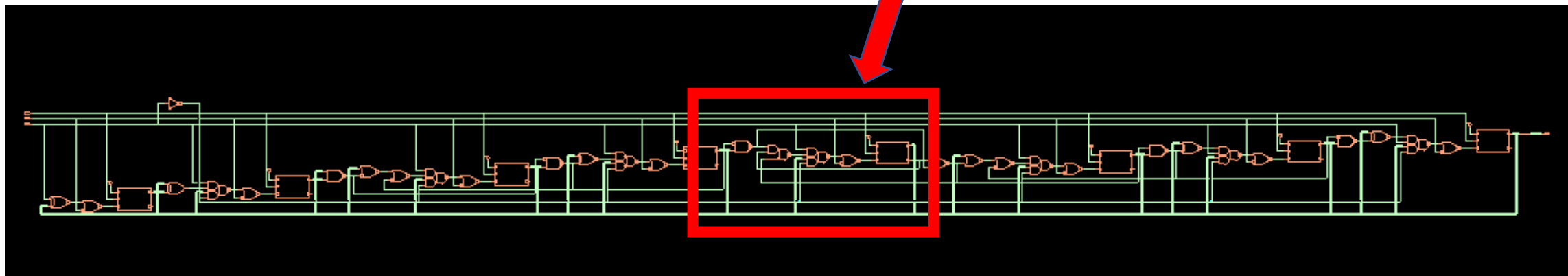
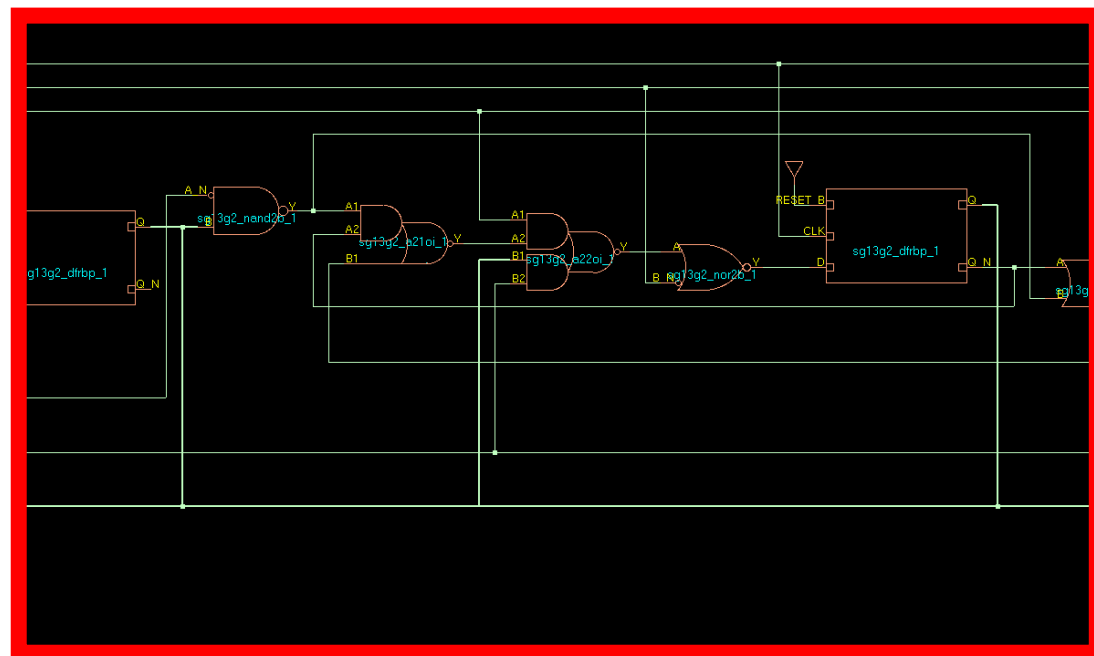
Результат: ячейки и соединения между ними

Размещение и трассировка — на следующих этапах. Название ячейки в примере условное.

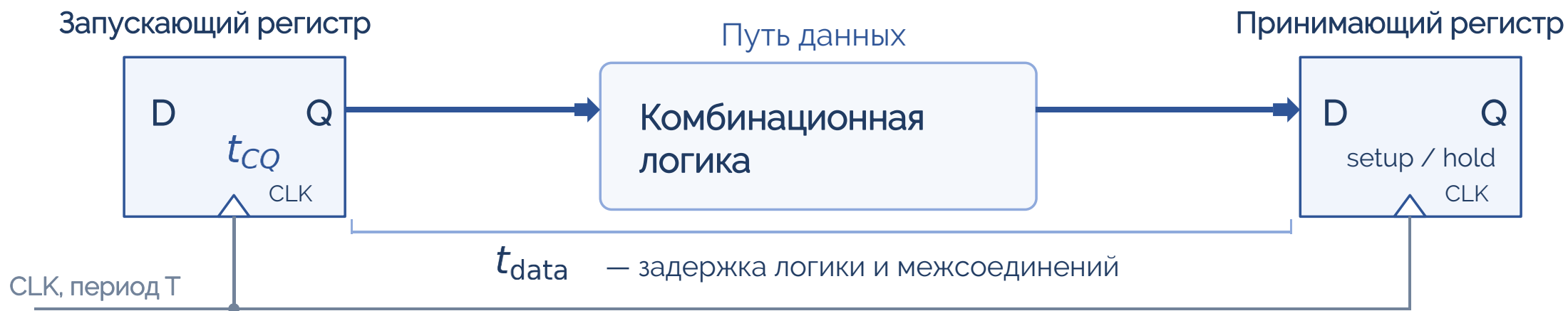


Что делает Логический синтез

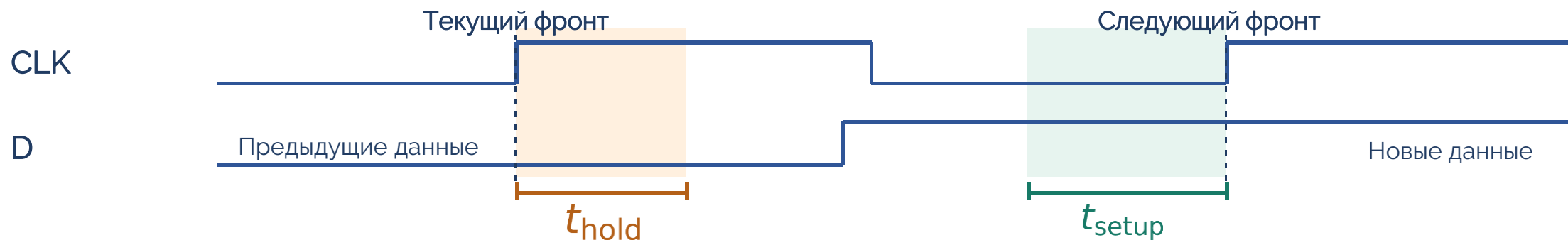
```
module counter (  
    input logic      clk,  
    input logic      rst_n,  
    input logic      enable,  
    output logic [7:0] count  
);  
  
    always_ff @(posedge clk) begin  
        if (!rst_n)  
            count <= 8'd0;  
        else if (enable)  
            count <= count + 8'd1;  
    end  
  
endmodule
```



Данные проходят от выхода Q запускаящего регистра к входу D принимающего



Hold проверяем на текущем такте, setup — на следующем



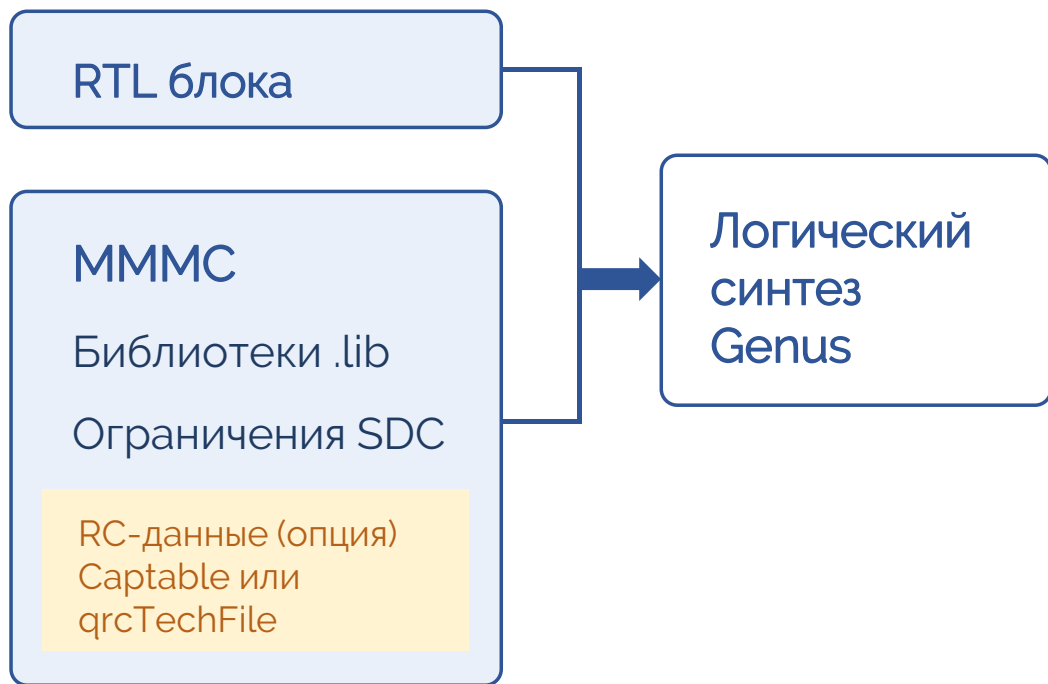
$$t_{CQ, \max} + t_{data, \max} + t_{setup} \leq T$$

$$t_{CQ, \min} + t_{data, \min} \geq t_{hold}$$

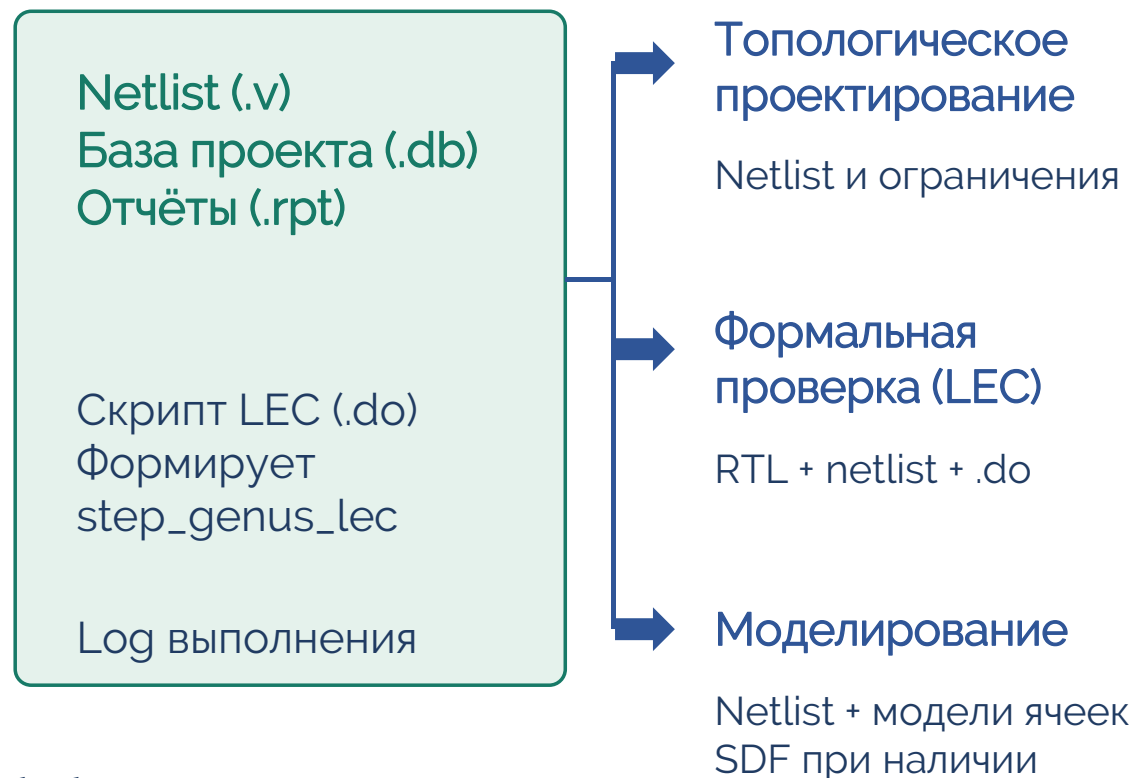
Упрощённый случай: один clock, активный нарастающий фронт, skew = 0, uncertainty = 0.

Что требуется для синтеза и где используются полученные файлы

Входные данные



Результаты



МММС задаёт режимы и углы анализа и подключает библиотеки и ограничения.



Liberty (.lib) описывает логические, временные и энергетические характеристики ячеек

library (...)

Общие параметры библиотеки

Условия: t_{up} , 1,2 В, 25 °С. Единицы: нс и пФ.

cell (sg13g2_inv_1)

Описание ячейки

Площадь, мощность утечки, данные энергопотребления.

pin (A), pin (Y)

Описание выводов

Направление, логическая функция, входная ёмкость.

timing ()

Временные дуги и таблицы

related_pin связывает вход с выходом.

cell_rise / cell_fall: задержка.

rise_transition / fall_transition: выходной transition.

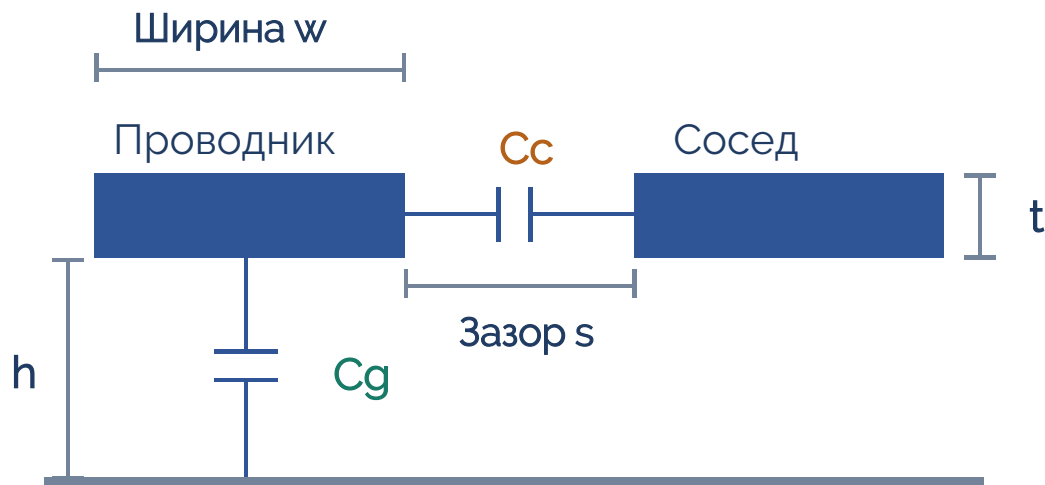
Для триггеров: clock-to-Q, setup и hold.

Для расчёта: входной transition t_{rvx} + нагрузка $C_{наг}$

По соответствующим таблицам находим задержку и выходной transition t_{rvx} .

Captable описывает сопротивления и ёмкости проводников для выбранной технологии

Два соседних проводника



Подложка / опорный проводник

L — длина вдоль проводника

Слои и геометрия

Данные для M1, M2 и других слоёв.
 t — толщина, h — расстояние до опоры.

Ёмкости

$$C_g \approx \frac{\epsilon w L}{h} \quad C_c \approx \frac{\epsilon t L}{s}$$

ϵ — диэлектрическая проницаемость.
Приближение плоских конденсаторов.
Captable учитывает краевые поля и окружение.

Сопротивление

$$R \approx \frac{\rho L}{wt} = R_{sh} \frac{L}{w}$$

ρ — удельное сопротивление, R_{sh} — сопротивление квадрата.

Как используется captable



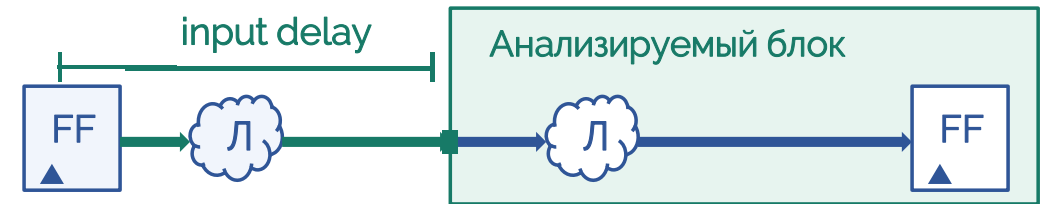
SDC задаёт требования по времени и описывает внешнее окружение блока.

Основные команды SDC

Команда	Что задаёт
create_clock	Период и фронты такта
set_clock_uncertainty	Неопределённость такта
set_input_delay	Приход данных на вход относительно такта
set_output_delay	Внешние временные требования к выходу
set_input_transition	Длительность фронта на входе
set_load	Ёмкостную нагрузку на выходе

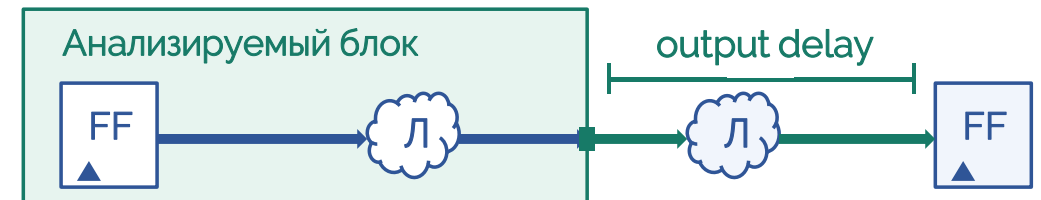
Внешняя часть пути между регистрами

Вход: внешний регистр запускает данные



input delay: clock → Q + задержка внешнего пути

Выход: внешний регистр принимает данные

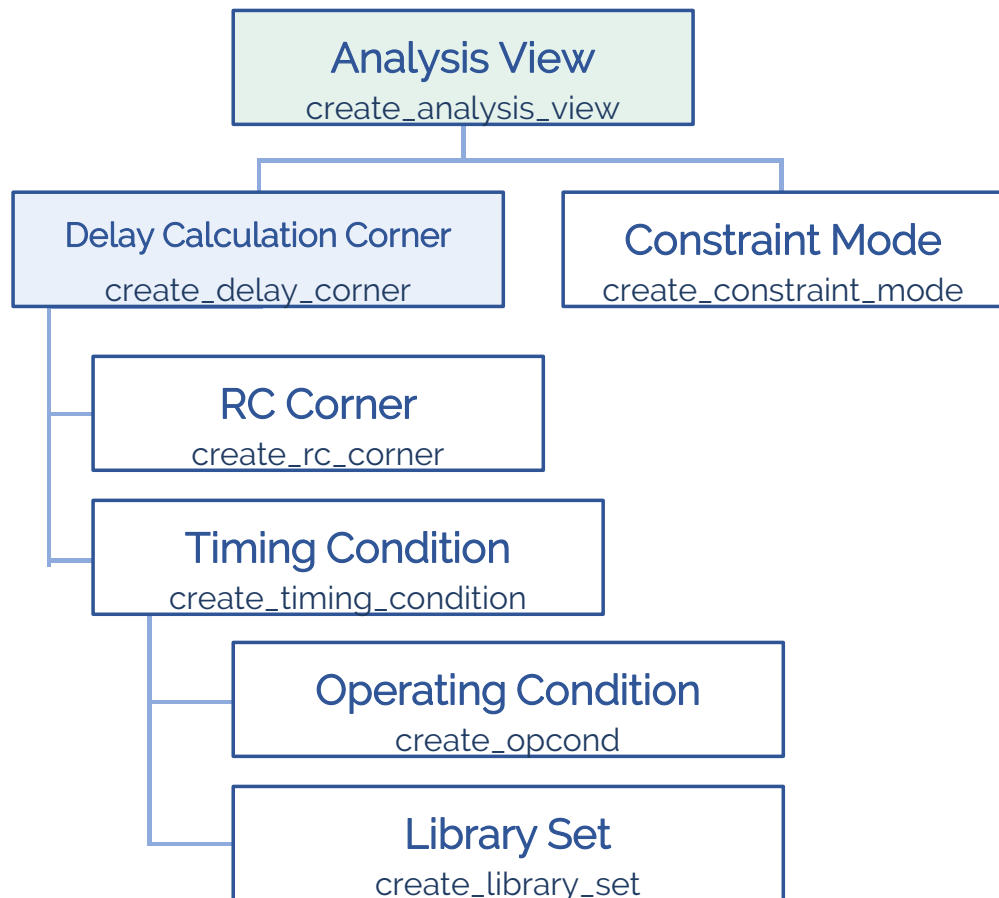


output delay: внешний путь + требования setup/hold

Для I/O: -max задаёт условия для setup, -min — для hold.

FF — регистр. Л — комбинационная логика. Зелёный цвет выделяет внешнюю часть пути.

Результат анализа зависит от ограничений и выбранных условий расчёта.



Analysis View

Сценарий проверки: Constraint Mode + Delay Corner.

Delay Calculation Corner

Условия расчёта задержек ячеек и межсоединений.

Constraint Mode

Набор SDC: такты, задержки входов/выходов, исключения.

RC Corner

Условия расчёта сопротивлений и ёмкостей проводов.

Timing Condition

Связывает Library Set с условиями работы ячеек.

Operating Condition

Процесс, напряжение и температура (PVT).

Library Set

Набор временных библиотек ячеек (.lib).

Analysis View = Constraint Mode (SDC) + Delay Corner (модели ячеек и RC)

Для выбранного сценария получаем результаты проверок setup/hold и запас времени (slack).



Ячейка sg13g2_inv_1: сокращённая выдержка из предоставленной .lib

```
cell (sg13g2_inv_1) {  
  area : 5.4432;  
  pin (Y) {  
    direction : "output";  
    function : "!(A)";  
    ...  
    timing () {  
      related_pin : "A";  
      timing_sense : negative_unate;  
      ...  
    }  
  }  
  pin (A) {  
    direction : "input";  
    capacitance : 0.0028869;  
  }  
}
```

Имя и площадь ячейки

sg13g2_inv_1 — выбранный инвертор.
area — площадь стандартной ячейки.

Вход A, выход Y

function : "!(A)" означает $Y = \neg A$.

Входная ёмкость

На входе A: 0,0028869 пФ $\approx$ 2,887 фФ.
Эту ёмкость видит предыдущая ячейка.

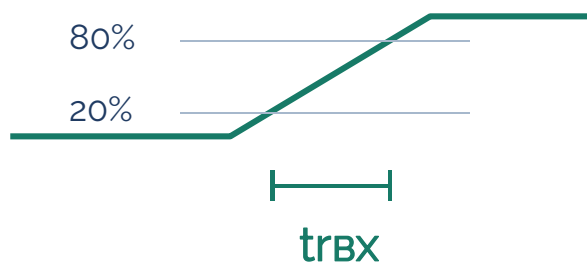
Временная дуга $A \rightarrow Y$

related_pin указывает связанный вход A.
negative_unate: $A \uparrow$ даёт $Y \downarrow$, $A \downarrow$ даёт $Y \uparrow$.

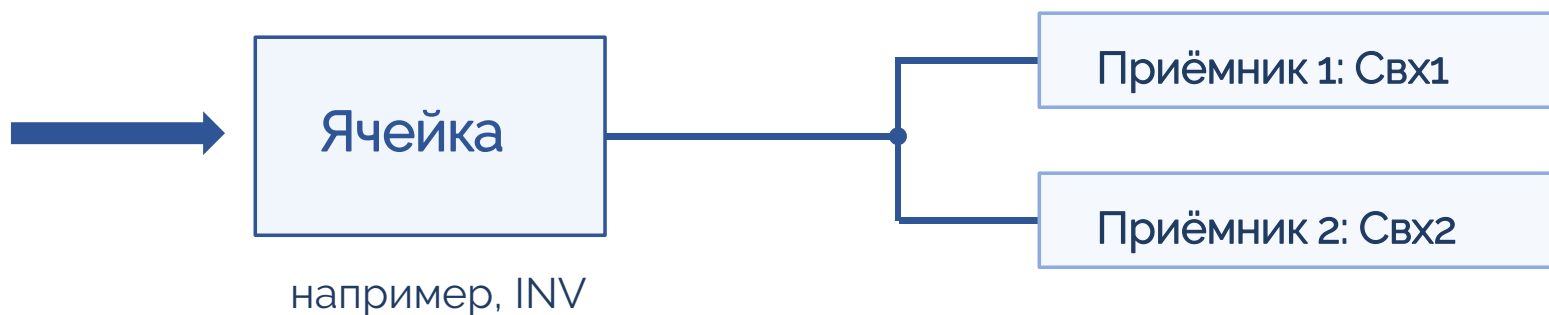
В cell_rise и cell_fall слова rise/fall относятся к переходу на выходе Y.

В таблицах нашей .lib задержка определяется входным transition и выходной нагрузкой

Входной transition trvx



Выходная нагрузка Cнаг



$$t_{\text{ячейки}} = F(trvx, C_{\text{наг}})$$

Для выбранной ячейки, временной дуги и PVT-условий

Что означает trvx

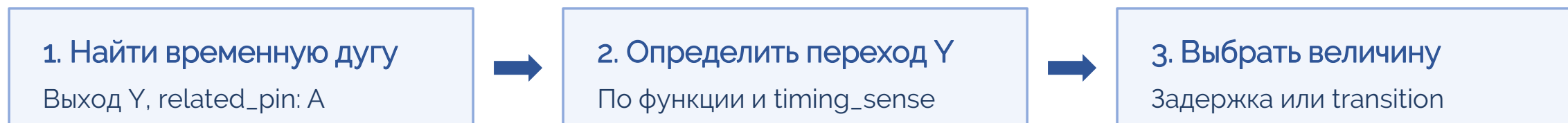
Время изменения входного сигнала между заданными уровнями напряжения.

Что входит в Cнаг

Входные ёмкости всех приёмников, ёмкость проводов и внешняя нагрузка.

PVT: вариант процесса, напряжение питания и температура. Задержку берём из cell_rise / cell_fall.

В названиях cell_rise / cell_fall слова rise и fall относятся к выходу ячейки



Для инвертора $A \rightarrow Y$: timing_sense : negative_unate

Переход на входе A	Переход на выходе Y	Задержка	Выходной transition
A $\uparrow$ (0 $\rightarrow$ 1)	Y $\downarrow$ (1 $\rightarrow$ 0)	cell_fall	fall_transition
A $\downarrow$ (1 $\rightarrow$ 0)	Y $\uparrow$ (0 $\rightarrow$ 1)	cell_rise	rise_transition

Затем используем trvx и Снаг в выбранной таблице.

В нашем .lib: index_1 — входной trvx, index_2 — выходная нагрузка Снаг.

Задержку и выходной transition рассчитываем отдельно.

В нашем шаблоне: index_1 — trvx (нс), index_2 — Cнаг (пФ)

Сокращённый фрагмент cell_fall

```
cell_fall (TIMING_DELAY_7x7ds1) {  
    index_1 ("0.0186, 0.0966, ...");  
    index_2 ("0.001, 0.0234, ...");  
    values (  
        "0.019836, 0.0720874, ...",  
        "0.0395964, 0.11771, ...",  
        ...  
    );  
}
```

Таблица задержек cell_fall, нс

trvx, нс ↓ Cнаг, пФ →	0.001	0.0234
0.0186	0.019836	0.0720874
0.0966	0.0395964	0.11771

trvx = 0,0186 нс, Cнаг = 0,0234 пФ

Задержка = 0,0720874 нс

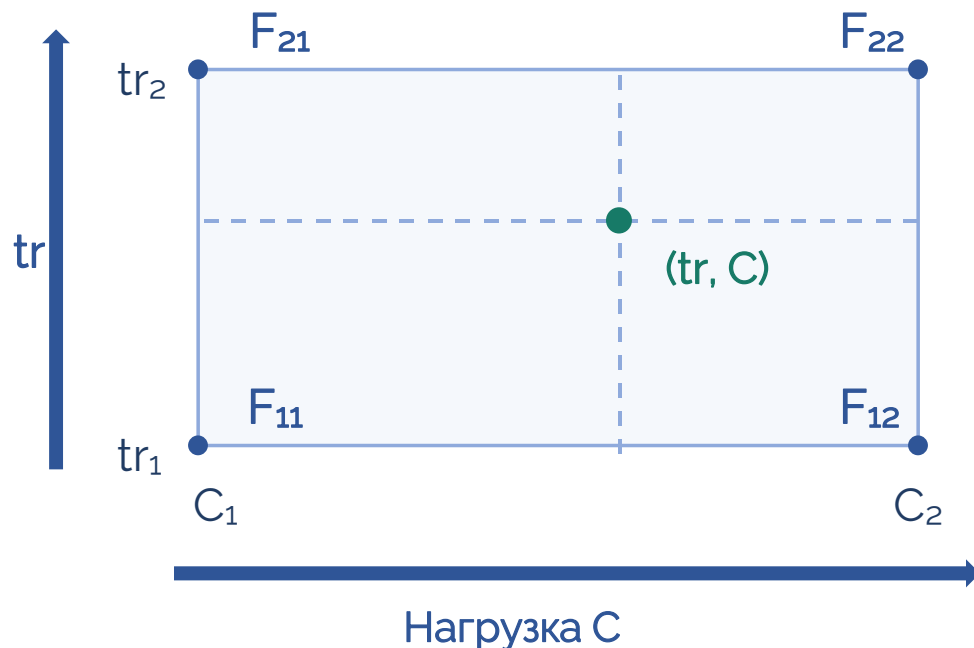
Каждая строка values соответствует одному значению index_1.

Числа внутри строки идут в порядке значений index_2.

Между узлами таблицы используем интерполяцию. Здесь значения совпадают с узлами.

Если tr и C находятся между узлами таблицы, используем четыре соседних значения

1. Выбрать четыре опорные точки



2. Найти положение точки

$$a = \frac{tr - tr_1}{tr_2 - tr_1}$$

a — положение по входному transition

$$b = \frac{C - C_1}{C_2 - C_1}$$

b — положение по выходной нагрузке

При интерполяции: $0 \leq a \leq 1$, $0 \leq b \leq 1$.

$F_{ij} = F(tr_i, C_j)$: задержка или transition из выбранной таблицы.

3. Последовательно выполнить интерполяцию

По C :

$$F_1 = F_{11} + b(F_{12} - F_{11}),$$

$$F_2 = F_{21} + b(F_{22} - F_{21})$$

По tr :

$$F(tr, C) = F_1 + a(F_2 - F_1)$$

Расчёт повторяем отдельно для задержки и выходного transition.



Пример расчета инвертора

Задано: на входе INV2 нарастание, $tr_{вх} = 0,08$ нс



Снаг = $C_{вх}(INV3) = 0,0028869$ пФ. Ёмкость и задержка проводов равны нулю.

sg13g2_inv_1: опорные значения (пс, фФ)

$tr_{вх}$, пс	cell_fall $C_1 = 1$	cell_fall $C_2 = 23,4$	fall_transition $C_1 = 1$	fall_transition $C_2 = 23,4$
$tr_1 = 18,6$	19,836	72,0874	10,3362	75,1289
$tr_2 = 96,6$	39,5964	117,710	25,1781	88,8987

$$a = \frac{80 - 18,6}{96,6 - 18,6} \approx 0,78718$$

$$b = \frac{2,8869 - 1}{23,4 - 1} \approx 0,08424$$

По C: $F_1 = F_{11} + b(F_{12} - F_{11}), \quad F_2 = F_{21} + b(F_{22} - F_{21})$

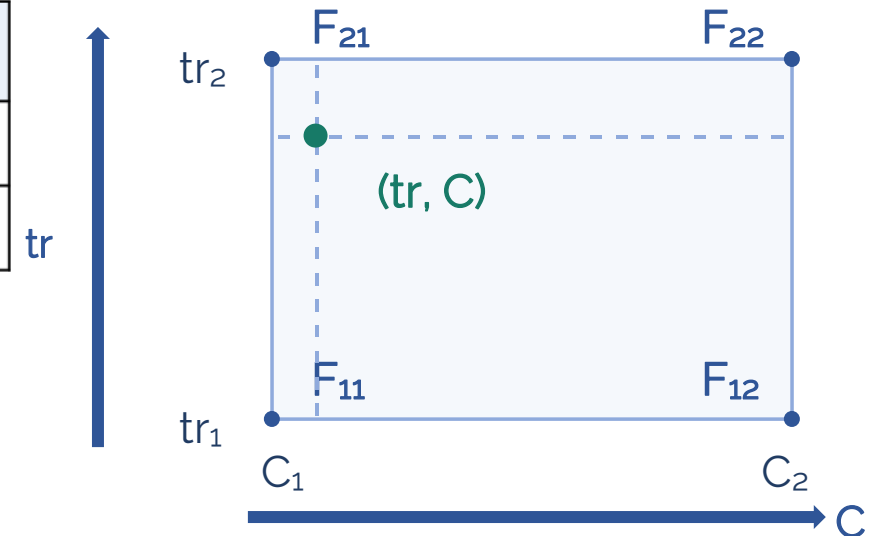
$$d_1 = 24,2375, d_2 = 46,1764 \text{ пс}$$

$$d = d_1 + a(d_2 - d_1) \approx 41,51 \text{ пс} = 0,04151 \text{ нс}$$

$$tr_{1вх} = 15,7941, tr_{2вх} = 30,5457 \text{ пс}$$

$$tr_{вых} = tr_{вых,1} + a(tr_{вых,2} - tr_{вых,1}) \approx 27,41 \text{ пс} = 0,02741 \text{ нс}$$

Четыре опорные точки



$$tr = 80 \text{ пс}$$

$$C = 2,8869 \text{ фФ}$$



```
# -----  
# Host info  
# -----  
if {[file exists /proc/cpuinfo]} {  
    sh grep "model name" /proc/cpuinfo  
    sh grep "cpu MHz" /proc/cpuinfo  
}  
puts "Hostname : [info hostname]"  
  
# -----  
# Global options  
# -----  
set_db timing_report_time_unit ns  
  
# CPUs / threading  
# set_db elaboration_threads      8  
# set_db synthesis_threads       8  
set_db max_cpus_per_server      8  
  
# Synthesis effort  
set_db syn_generic_effort       medium  
set_db syn_map_effort           medium  
set_db syn_opt_effort           medium  
  
# Optimization controls  
set_db information_level        1  
  
# HDL  
set_db init_hdl_search_path     $::env(ENV_INIT_HDL_SEARCH_PATH)  
  
# Choose one options:  
# read_hdl -define $::env(ENV_DEFINE) -language v2001 -f $::env(ENV_RTL_LIST) ; # if rtl write on Verilog  
read_hdl -define $::env(ENV_DEFINE) -sv -f $::env(ENV_RTL_LIST) ; # if rtl write on SystemVerilog
```

Диагностика сервера

Host info выводит модель и частоту CPU, а также имя узла в log.

Единицы и ресурсы

Время в отчётах задаётся в нс.

max_cpus_per_server ограничивает число CPU.

Параметры синтеза

Уровень medium задаёт усилие алгоритмов для syn_generic, syn_map и syn_opt.

Подробность сообщений

information_level управляет количеством информационных сообщений инструмента.

Пути поиска RTL

init_hdl_search_path задаёт каталоги, в которых Genus ищет исходные HDL-файлы.

Чтение исходного описания

read_hdl читает список RTL-файлов и defines. Ключ -sv включает SystemVerilog.

Две строки read_hdl — альтернативы: выберите одну по языку RTL.



```
read_mmmc $::env(ENV_MMMC)

elaborate $::env(ENV_DESIGN)
check_design -unresolved > $::env(GENUS_RUN_DIR)/check_design.rpt

init_design
check_timing_intent > $::env(GENUS_RUN_DIR)/check_timing_intent.rpt

set STAGE syn_generic
puts "\033\]2;$STAGE\a"

syn_generic

set STAGE syn_map
puts "\033\]2;$STAGE\a"

syn_map

set STAGE syn_opt
puts "\033\]2;$STAGE\a"

syn_opt

set dir $::env(GENUS_REPORT_DIR)/${STAGE}

report_timing > ${dir}/report_timing.rpt
report_timing_summary > ${dir}/report_timing_summary.rpt
report_area > ${dir}/report_area.rpt
report_power > ${dir}/report_power.rpt
report_qor > ${dir}/report_qor.rpt
report_hierarchy > ${dir}/report_hierarchy.rpt
check_timing_intent -verbose > ${dir}/check_timing_intent.rpt
}

set dir $::env(GENUS_OUTPUT_DIR)/${STAGE}

write_hdl > ${dir}/${::env(ENV_DESIGN)}.v
write_db -to_file ${dir}/${::env(ENV_DESIGN)}.db

write_do_lect -golden_design rtl -revised_design $output_netlist -log_file $::env(GENUS_REPORT_DIR)/rtl2final.lect.log \
> $::env(GENUS_OUTPUT_DIR)/rtl2final.lect.do

gui_show
```

Подключение условий анализа

read_mmmc читает конфигурацию MMMC: сценарии, библиотеки и ограничения.

Построение проекта из RTL

elaborate строит иерархию выбранного модуля.
check_design проверяет неразрешённые ссылки.

Инициализация и проверка ограничений

init_design подготавливает проект к синтезу.
check_timing_intent проверяет ограничения.

Преобразование и оптимизация

syn_generic создаёт обобщённую схему.
syn_map выбирает ячейки, syn_opt оптимизирует.

Отчёты и выходные файлы

report_* сохраняют результаты анализа.
write_hdl пишет netlist, write_db — базу проекта.

Подготовка проверки эквивалентности

write_do_lect создаёт скрипт для Conformal: сравнение исходного RTL с итоговым netlist.

LEC выполняется отдельно в Conformal. gui_show открывает просмотр схемы.

Определите количество ячеек и их суммарную площадь после syn_opt.

Пример отчёта для блока counter

```
=====
Generated by:      Genus(TM) Synthesis Solution 19.15-s090_1
Generated on:      Oct 05 2026  01:49:28 pm
Module:           counter
Operating conditions: sg13g2_stdcell_typ_1p20V_25C (balanced_tree)
Wireload mode:     top
Area mode:         timing library
=====

Instance Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
counter          40      696.465    297.552    994.017     0_5k (S)

(S) = wireload was automatically selected
```

40

Cell Count — число ячеек

696,465

Cell Area — площадь ячеек

Cell Count

Количество экземпляров ячеек
в выбранном блоке.

Cell Area

Сумма площадей ячеек.
Этот показатель нужен для задания.

Net Area

Оценка площади межсоединений.
В примере используется wireload-модель.

Total Area

Cell Area + Net Area.
В примере: $696,465 + 297,552 = 994,017$.

Wireload

Модель межсоединений: 0_5K.
(S) — выбрана автоматически.

Единицы площади проверьте в библиотеке: в этом фрагменте они не указаны.

Площадь ячеек не равна площади кристалла: размещение и физическая реализация ещё впереди.



```

=====
Generated by:      Genus(TM) Synthesis Solution 19.15-s090_1
Generated on:      Oct 05 2026 01:49:28 pm
Module:           counter
Operating conditions: sg13g2_stdcell_typ_1p20V_25C (balanced_tree)
Wireload mode:    top
Area mode:        timing library
=====

Path 1: MET (7.626 ns) Setup Check with Pin count_reg[7]/CLK->D
  View: typ
  Group: clk
  Startpoint: (F) enable
  Clock: (R) clk
  Endpoint: (R) count_reg[7]/D
  Clock: (R) clk

      Capture      Launch
      Clock Edge: + 10.000      0.000
      Drv Adjust: + 0.000      0.000
      Src Latency: + 0.000      0.000
      Net Latency: + 0.000 (I)  0.000 (I)
      Arrival: = 10.000      0.000

      Setup: - 0.120
      Required Time: = 9.880
      Launch Clock: - 0.000
      Input Delay: - 2.000
      Data Path: - 0.255
      Slack: = 7.626

Exceptions/Constraints:
  input_delay      2.000      counter.sdc_line_11_1_1

#-----
# Timing Point  Flags  Arc  Edge  Cell  Fanout  Load  Trans  Delay  Arrival  Instance
#              (fF)  (ns)  (ns)  (ns)  Location
#-----
enable         -      -   F   (arrival)  9 32.3 0.000 0.000 2.000 (-,-)
g237/Y         -      A->Y R   sg13g2_inv_1  7 37.2 0.155 0.113 2.113 (-,-)
g203_6260/Y    -      B2->Y F  sg13g2_a22oi_1  1 3.7 0.062 0.085 2.198 (-,-)
g200_2398/Y    -      A->Y R   sg13g2_nor2b_1  1 2.4 0.047 0.057 2.255 (-,-)
count_reg[7]/D <<< -      -   R   sg13g2_dfrbp_1  1 - - 0.000 2.255 (-,-)
#-----

```

Какой путь проверяется

Setup: вход enable → count_reg[7]/D.

View: typ; группа путей: clk.

Required Time = 9,880 нс

Предельное время прихода данных.

Фронт захвата: 10 нс; setup: 0,120 нс.

Input Delay = 2,000 нс

Задержка вне анализируемого блока,
заданная для входа enable.

Data Path = 0,255 нс

Задержка пути внутри блока.

Приход в D: 2,000 + 0,255 = 2,255 нс.

Slack = +7,626 нс

MET — setup для этого пути выполнен.

$$\text{Slack}_{\text{setup}} = t_{\text{required}} - t_{\text{arrival}}$$

Один путь не описывает всю схему: общий вывод делайте по
timing_summary.



```
=====
Generated by:      Genus(TM) Synthesis Solution 19.15-s090_1
Generated on:      Oct 05 2026 01:49:28 pm
Module:            counter
Operating conditions: sg13g2_stdcell_typ_1p20V_25C (balanced_tree)
Wireload mode:     top
Area mode:         timing library
=====

# SETUP                WNS      TNS      FEP
# -----
View: ALL              7.626    0.000      0
  Group : in2out        N/A      N/A      N/A
  Group : in2reg        7.626    0.000      0
  Group : reg2out       7.770    0.000      0
  Group : reg2reg       8.760    0.000      0

# -----
# DRV                  WNS      TNS      FEP
# -----
View: ALL
  Check: max_transition  N/A      N/A      0
  Check: max_capacitance N/A      N/A      0
  Check: max_fanout     N/A      N/A      0

# -----
```

Группы путей

in2reg — вход → регистр; reg2out — регистр → выход

reg2reg — регистр → регистр; in2out — вход → выход

Hold здесь не показан. Для полного вывода проверьте остальные проверки и полноту ограничений.

WNS — наихудший slack

Самый малый запас в группе.

Отрицательное значение — нарушение.

TNS — сумма отрицательных slack

Характеризует суммарные нарушения.

В примере: 0,000 нс.

FEP — Failing Endpoints

Число конечных точек с нарушениями.

В примере: 0.

DRV — ограничения для сигналов

max_transition, max_capacitance, max_fanout.

N/A — численный результат не представлен.

SETUP: WNS = +7,626 нс

TNS = 0; FEP = 0.

Setup по отчёту выполнен.



Передовые
инженерные
школы

Проверка эквивалентности



МИЭТ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Настройте свой блок в Makefile. Команды make запускайте из каталога с этим файлом.

Настройки проекта

```
# =====  
# Design settings  
# =====  
  
export ENV_DESIGN          = counter  
export ENV_DESIGN_DIR     = $(PWD)/data/example_rtl/counter  
export ENV_MMMC            = $(ENV_DESIGN_DIR)/mmmc.view  
export ENV_DEFINE         =  
export ENV_INIT_HDL_SEARCH_PATH = ${ENV_DESIGN_DIR}  
export ENV_RTL_LIST        = $(ENV_DESIGN_DIR)/counter.lst  
# =====
```

Модуль и RTL-файлы

ENV_DESIGN, ENV_DESIGN_DIR, ENV_RTL_LIST

Условия анализа

ENV_MMMC: путь к конфигурации MMMC

Параметры чтения RTL

ENV_DEFINE: макроопределения

ENV_INIT_HDL_SEARCH_PATH: пути поиска

Genus: логический синтез

module load cadence/GENUS/21.14

make genus

Интерактивный запуск

make synth

Выполнение flow-скрипта

Conformal: проверка LEC

module load cadence/CONFRML/23.10.100

make lec

Запуск с GUI

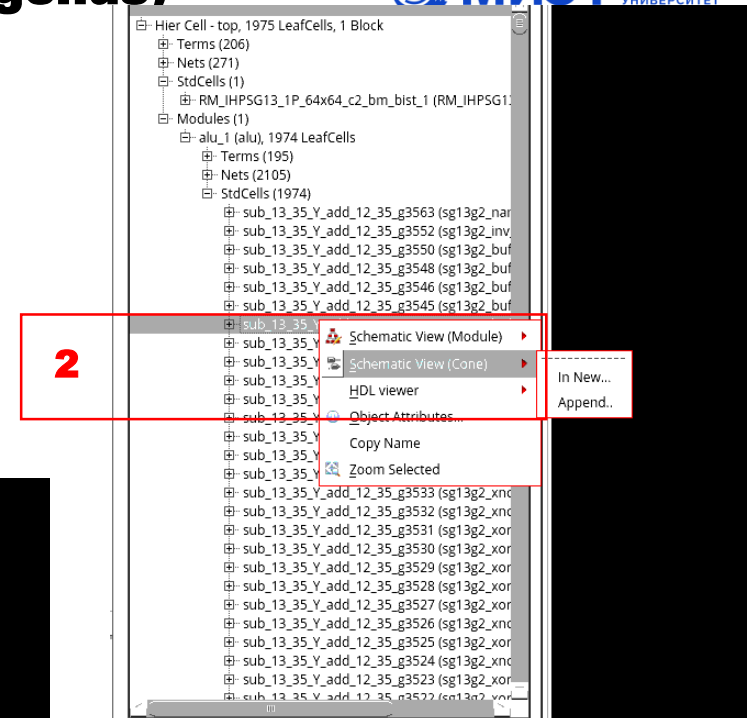
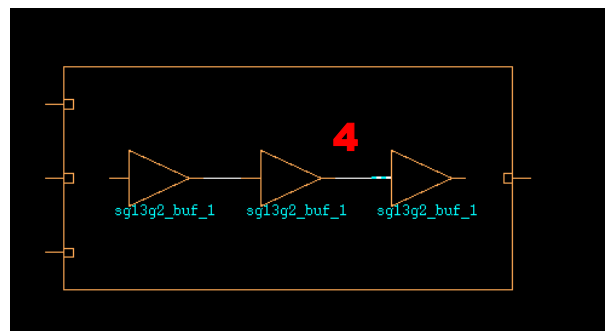
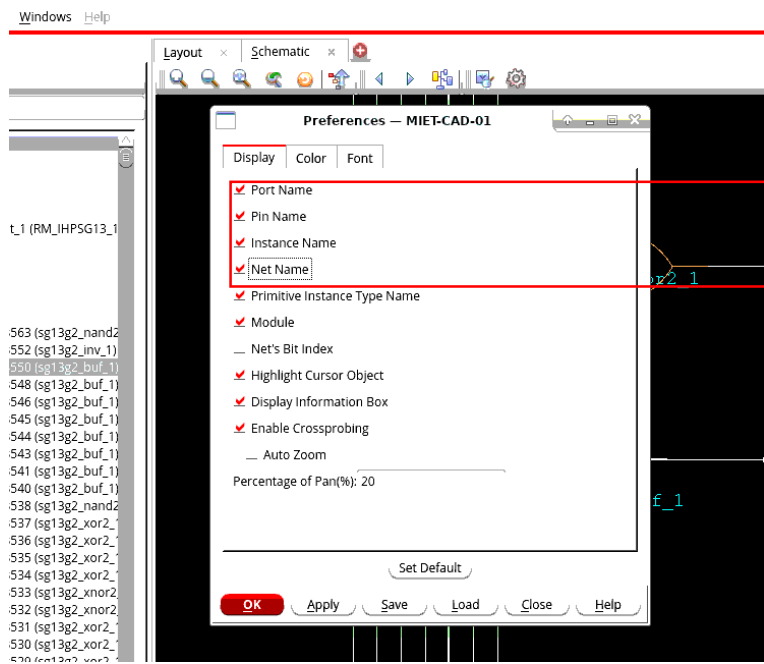
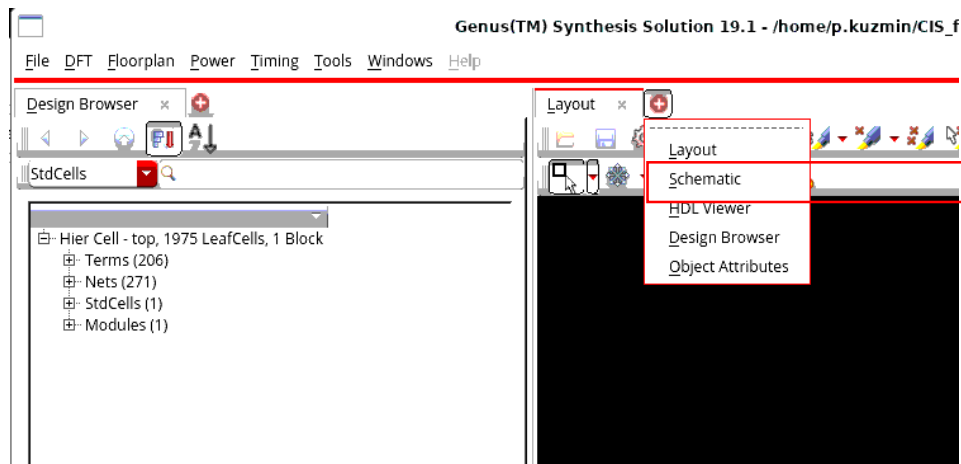
make lec_nogui

Запуск без GUI

Перед LEC: получите итоговый netlist и сформируйте rtl2final.lec.do в Genus.



Полезные команды в gui режиме (genus)



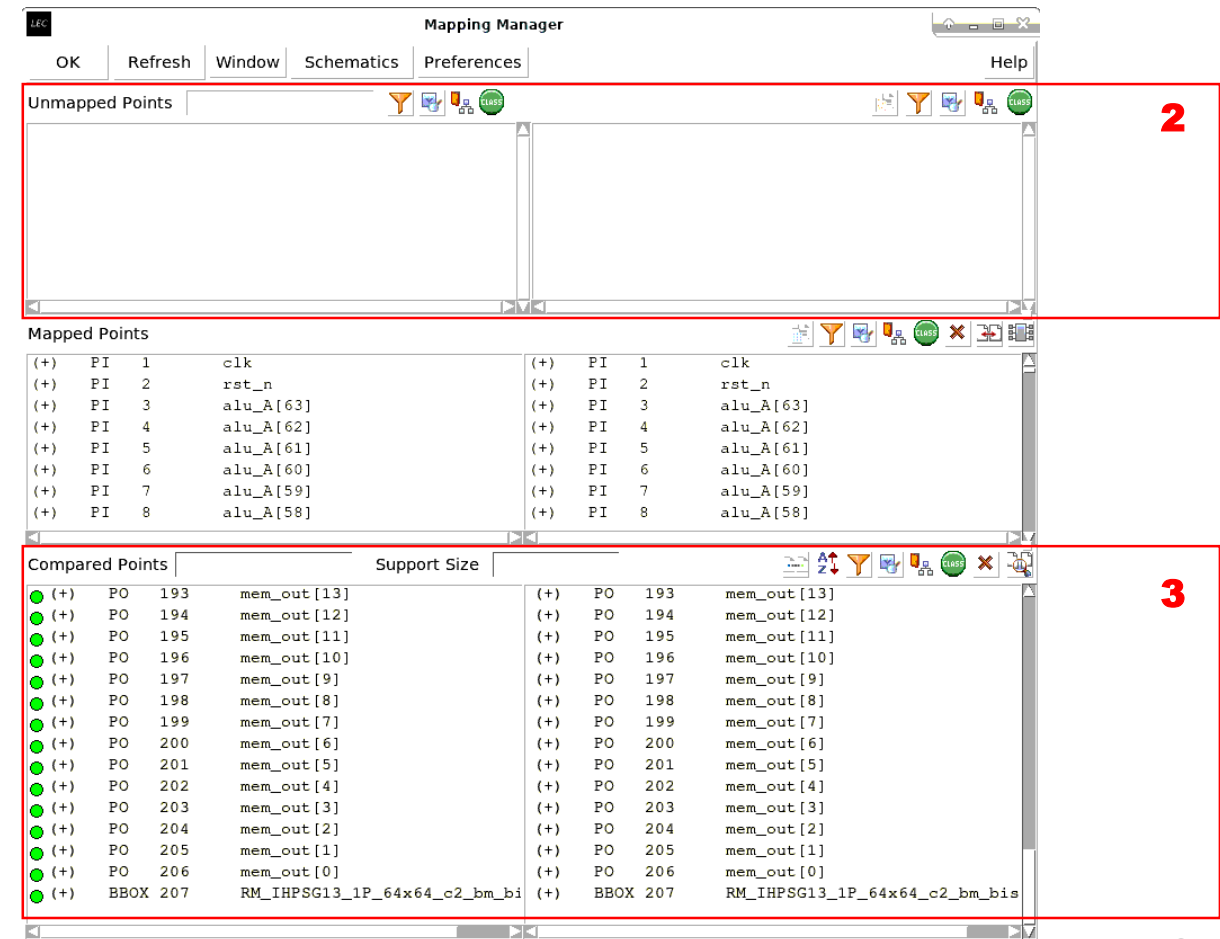
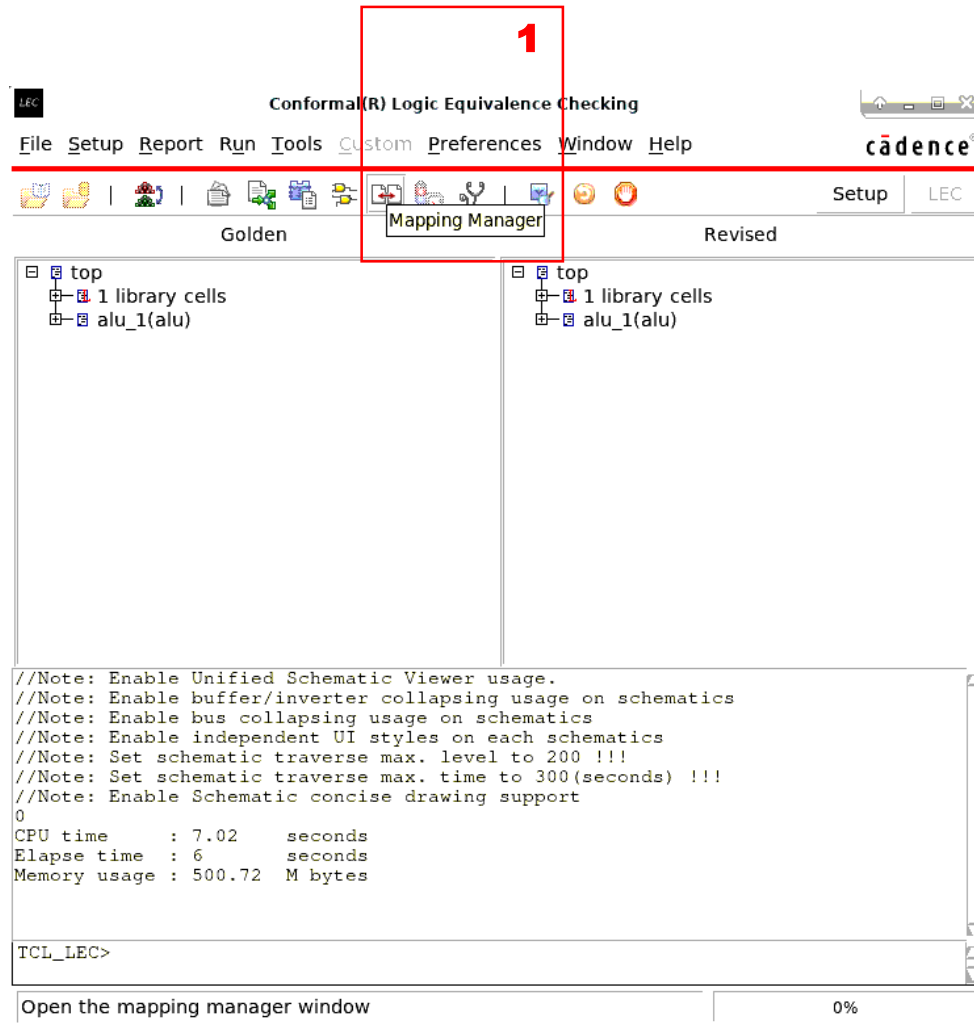
- 1) Переход на вид schematic
- 2) Открыть определенный элемент как schematic
- 3) Отображение названий port, pin, instance, net
- 4) Если нажать 2 раза на вход или выход элемента, то раскроется путь



Полезные команды в gui режиме (conformal)

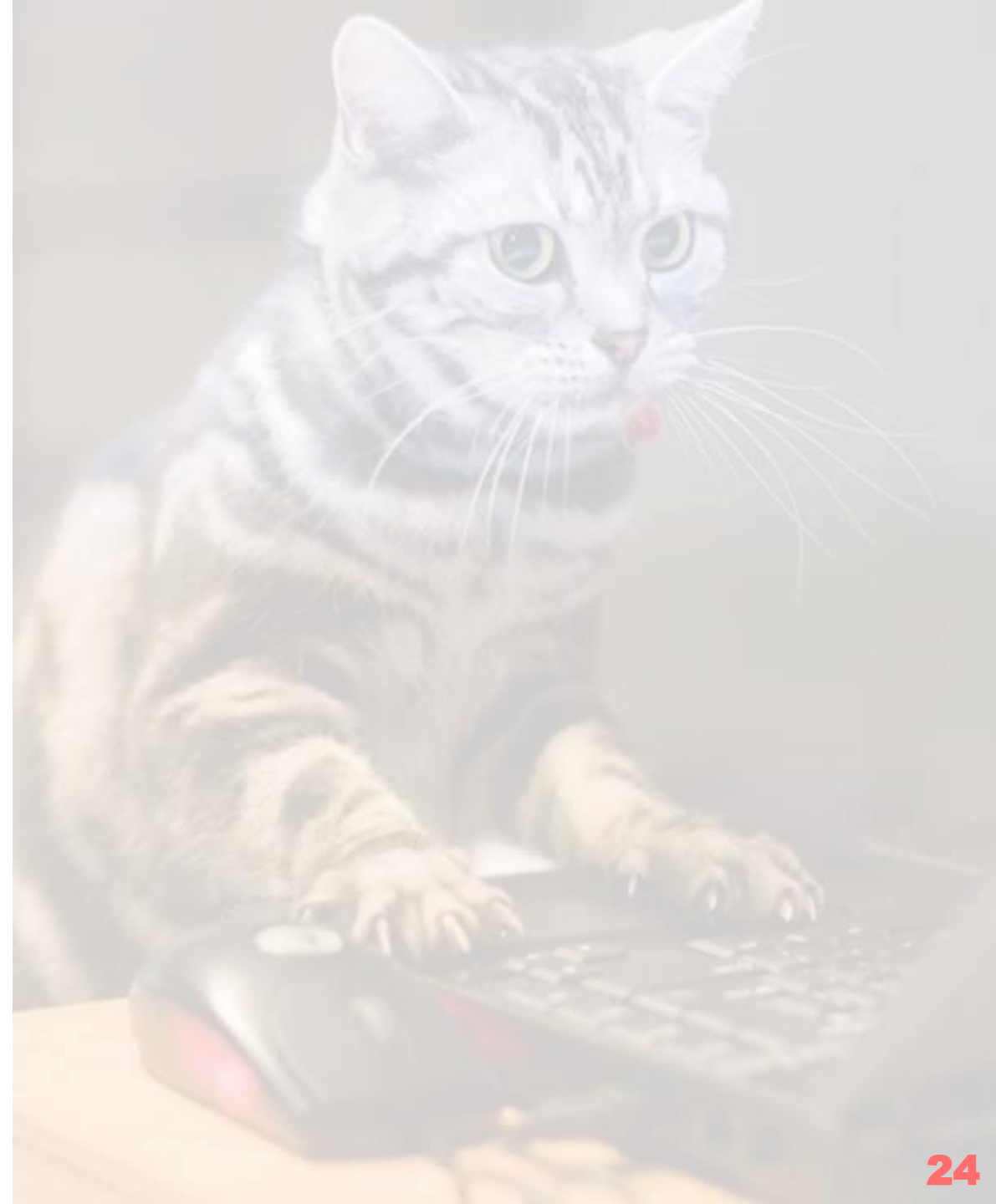


- 1) Открывает mapping manager
- 2) Элементы которые отличаются в RLT и netlist (смотри где конкретно)
- 3) Если произошла ошибка при сравнении, то появится красный круг



Лабораторное задание

Подробности смотрите в файле README



Используйте проверенный RTL своего блока из лабораторной работы №1.

Преподаватель предоставляет: steps.genus.tcl, библиотеку .lib, MMMC и SDC.

Что нужно сделать

- 01** **Подготовить проект**
Указать свой блок как top и настроить список RTL-файлов.
- 02** **Составить управляющий Tcl-скрипт**
Подключить steps.genus.tcl, выбрать порядок вызовов и добавить комментарии к основным этапам.
- 03** **Выполнить синтез в Genus и разобрать результат**
Оценить состав и число ячеек, площадь, временной путь и slack по итоговым отчётам syn_opt.
- 04** **Проверить эквивалентность в Conformal**
Сформировать LEC-скрипт и подтвердить эквивалентность исходного RTL и итогового netlist.
- 05** **Провести эксперимент с функциональной ошибкой**
Изменить отдельную копию RTL, повторить LEC и объяснить, какие точки сравнения не прошли проверку.

Для следующих лабораторных сохраните корректный netlist, успешно прошедший LEC.

На защиту

RTL и Tcl-скрипты
Итоговый netlist
Log синтеза и отчёты
LEC-скрипт и результаты
успешной проверки
Изменённый RTL и результат
проверки с ошибкой

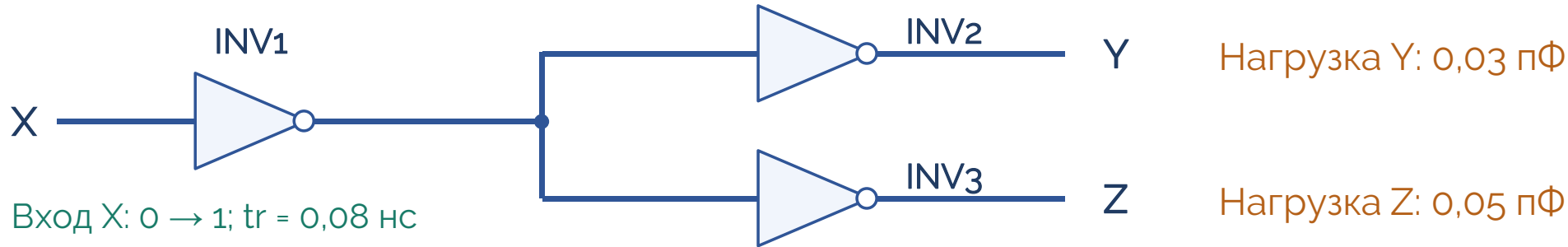
Будьте готовы объяснить

Этапы flow и смысл отчётов.
Оба результата LEC.



Рассчитайте задержки и выходные transition для двух путей схемы.

Все инверторы — sg13g2_inv_1. Ёмкости и задержки межсоединений равны нулю.



Что нужно сделать

- 01** Определить переходы и нагрузки
Для INV1 учесть входные ёмкости обоих приёмников.
- 02** Выбрать таблицы и выполнить интерполяцию
Для каждого элемента найти задержку и выходной transition.
- 03** Передать transition следующему элементу
Выходной transition INV1 — входной для INV2 и INV3.
- 04** Получить задержки путей $X \rightarrow Y$ и $X \rightarrow Z$
Сложить задержки элементов каждого пути.

На защиту

Таблица расчёта INV1–INV3
Таблица задержек двух путей
Промежуточные вычисления
Объяснение влияния
ответвления на оба пути

Задержка и выходной transition — разные величины: рассчитывайте их по отдельным таблицам.

Выберите удобный формат защиты. В обоих случаях проводится беседа с преподавателем.

01

С подготовленным отчётом

Заранее оформите результаты и выводы в отчёте.

На защите объясните ход работы и ответьте на вопросы.

02

Без отчёта

Покажите сохранённые файлы и результаты работы.

На защите объясните ход работы и ответьте на вопросы.

Требования одинаковые для обоих форматов

Выполните все задания и будьте готовы объяснить свои действия и полученные результаты.